EPSR-test-ex45 readpdb: triclinic cell from stdin, no dihedrals

Program(s): readpdb | chained passes: 1 | verification: ALL IDENTICAL (1 pass)

What this case tests

Exercises the cell and input-validation branches of `readpdb` [1,2]: a valid triclinic `CRYST1` record followed by one whose angle field cannot be read, so the cell is asked on standard input; an invalid van der Waals factor replaced by the default; no dihedral groups; one molecule per residue without `CONECT` records.

Code exercised

Fortran (original, fortran_src/):

- `readpdb.f`: `run_readpdb`, `readpdb` (CRYST1/ATOM/CONECT records, molecules from residues, bonds from van der Waals radii, angle constraints, DIH groups), `upcase`
- `ato_manipulation_rev4.f`: `ctype`, `ftype`, `centreofmass`, `get_rotgrp`, `dihedral`
- `auxiliary_routines.f`: `convertalphabetagamma`; `ato_read_and_save.f`: `moloutc`

Rust (port, rust/epsr-core/src/):

- `tools/readpdb.rs`: `run`, `readpdb`, `conect` (molecule merging), `make_dihedrals`, `read_fixed`, `upcase`
- `tools/readxyz.rs`: `set_type_parameters`; `ato.rs`: `State::ctype`, `State::ftype`
- `tools/groupc_common.rs`: `get_rotgrp`, `dihedral`, `convertalphabetagamma`, `setup_lattice`, `centreofmass`, `moloutc` (with `DIH` groups)

Physical system

Four molecules (23 atoms) in a triclinic cell a, b, c = 21, 23.5, 26.25 Å, alpha, beta, gamma = 80°, 96°, 110° (EPSR cell angles 110°, 10.380°, 125.462° in the header): a water, a dimethyl sulphoxide, a TIP4P-like water whose dummy site becomes a `Q` atom, and a methanol. Bonds are found where the distance is below 0.7 × the sum of the van der Waals radii, and bond-angle constraints are generated.

Inputs

`cluster.pdb` from a helper script (a one-off script that was not kept; the generated file is the input); the second `CRYST1` record has angle `8x.00`. `readpdb` `cluster` < `readpdb.stdin` (no extension): cell `21.0` `23.5` `26.25` `80.0` `96.0` `110.0`, factor `0` (invalid -> 0.7), dihedrals `no`. `vanderWaalsRadii.txt` copied from EPSR-test.

How the accuracy was verified

`commands.txt` (1 program call, run once) was run with the Fortran reference build (`rust/tools/build_fortran_reference.sh`: the original sources, gfortran -O2 -fno-tree-vectorize, single thread) and with the Rust `epsr-rs` on identical copies of the inputs, and every file written was compared byte for byte (`rust/tools/run_ex_test.sh`, `compare_outputs.py`; no masking except where noted below). The Fortran output is stored in `fortran_reference.tar.gz`. `readxyz`/`readpdb` crash (SIGSEGV) as compiled from the original source, so the reference build adds an explicit INTERFACE block for `ctype` (`rust/tools/build_patched_readxyz_readpdb.sh`); the port behaves like the patched program. Result of the final regression run of the complete port: every file identical in 1 pass.
Output compared: `cluster.ato`. Fortran reference: the original `readxyz`/`readpdb` binaries crash (SIGSEGV) at the first call of `ctype`, which has optional arguments but is called with 11 of its 13 arguments without an explicit interface (gfortran passes the hidden character lengths where `radmaxsph`/`pseudofacsph` are expected). The reference was therefore produced with an explicit `INTERFACE` block for `ctype` added to the program (no other change; absent arguments mean `radmaxsphere` = `0.5*sigcor`, `pseudofacmol` = `1`), built by `rust/tools/build_patched_readxyz_readpdb.sh`; the comparison is against this patched build, not the original binary.

References

[1] wwPDB, "Atomic Coordinate Entry Format Description", version 3.3 (2012), https://www.wwpdb.org/documentation/file-format.
[2] A. K. Soper, "Empirical Potential Structure Refinement - EPSRshell: a user's guide", Technical Report RAL-TR-2011-012, STFC Rutherford Appleton Laboratory (2011).
[3] GNU Fortran (gfortran) 15.2, GCC, https://gcc.gnu.org/fortran/ - reference build with -O2 -fno-tree-vectorize.